UNIVERSITY OF CAMBRIDGE
Department of Computer Science and Technology

May 15, 2026

Efficient Coinductives through State-Machine Corecursors

William Sørensen

Gonville & Caius College

Submitted in partial fulfilment of the requirements for the Computer Science Tripos, Part II

Declaration

I, William Sørensen of Gonville & Caius College, being a candidate for the course, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.
Signed: William Sørensen
Date: May 15, 2026

Proforma

Word count: 2323

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit.

Contents

1 Introduction ... 7
1.1 Use cases ... 7
1.1.1 Infinite datastructures ... 7
1.1.2 Compiler semantics ... 7
1.1.3 Safe webservers ... 7
2 Preparation ... 8
2.1 Dependent type theory ... 8
2.1.1 Categorically ... 8
2.2 Polynomial functors ... 8
2.2.1 Common pfunctors ... 9
2.2.2 Lean formalization ... 9
2.2.3 F-(co)algebras ... 10
2.3 Recursion and Inductives ... 10
2.3.1 Categorically ... 10
2.4 Corecursion and coinductives ... 10
2.4.1 Morally ... 11
2.4.2 In OCaml ... 11
2.4.3 Categorically ... 11
2.5 Tools used ... 11
2.6 Requirement Analysis ... 11
2.6.1 State machine encoding (Core) ... 11
2.6.2 Non-termination-monad (Core) ... 11
2.6.3 ABI Type (Extention) ... 11
2.6.4 Interaction Trees (Extention) ... 11
2.6.5 Futumorphic productivity (Extention) ... 11
2.6.6 Choice Trees (Extention) ... 11
3 Implementation ... 12
3.1 Transiteration ... 12
3.2 Stream implementation ... 13
3.3 Expanding the progressive approximation theory ... 13
3.3.1 Universe lifting of polynomial functors. ... 13
3.3.2 Generalizing the corecursor ... 13
3.4 State machine encoding ... 14
3.4.1 PreM ... 14
3.5 Interaction trees ... 14
3.6 The ABI Type ... 14
3.6.1 Weak univalence ... 16
3.6.2 Relation to Shrink and further universe transforming types ... 16
4 Evaluation ... 17
4.1 Performance between SME and PA ... 17
5 Conclusions ... 19

Bibliography 20
6 Appendicies 21
6.A Graphical p-functors 23
6.A.A Common pfunctors 24
7 Proposal 26

Chapter 1: Introduction

1.1 Use cases

1.1.1 Infinite datastructures

Often, a problem in programming requires leaving the finite. In cases we want to maintain safety properties of the language. For example:

- We cannot use termination as a critaria for infinite structures, so how do we maintain the totallity of the language.
- One would also want functions written for a language like this to have the expected runtime; we want a constant time to unfold.
- We want an interface that lets us unfold and reasona bout definition just how we would any other function.

A concreate example of a usecase for this would be reading from a UNIX file. Since we can for example have a program `cat` which echos out all it is fed, then what would the output of `yes | cat` be? This would have to be a stream.

1.1.2 Compiler semantics

Recalling L1[1], we gave it a big-step operational semantic. We later tried to implement an interpreter for this semantic to execute L1 programs. This required us to prove quite a few substantial coherences. An alternative way we could have tried would be to write down an interpreter, and simply use this as the semantic. Most proof-assistants would not allow this though, as they require function totallity. So we would want:

- A way to write an interpreter of a partial language, and be able to use this interpreter as a semantic in formal software.
- We would also want the asymptotics of the interpreter to be the same as the source language.
- We want to be able to encode effects in the langauge in addition to just non-termination.

1.1.3 Safe webservers

[1] TODO: Add reference to sem of prog

Chapter 2: Preparation

2.1 Dependent type theory

We can start by adding a type for Types, giving us a higher order logic. Recall back to discrete maths[2], how functions between sets $A, B : \mathbf{Type}$, $A \to B$ correspond with a form of arbitrary product, $B^A \equiv \prod_{x:A} B$. We can generalize this by changing B to be a function $B : A \to \mathbf{Type}$, then letting us write the Π-type[3] $\prod_{x:A} B(x)$. We can also do the dual construction, and get Σ-types. The inference rules for these can be seen in Figure 1. These rules should be familiar as they are the analogous to the rules for $\forall, \exists$ in the 1st order sequent calculus, their relationship can be seen in Equation 1. If you are unfamiliar with dependent types, I recommend viewing them using this relationship. For a more in depth at dependent type theory, read [1].

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\prod_{x:A} B(x)}\,\Pi\mathrm{f}$$

(a) Π-forming rule

$$\frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x.v : \prod_{x:A} B(x)}\,\Pi\mathrm{i}$$

(b) Π-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash f : \prod_{x:A} B(x) \\ \Gamma \vdash v : A \end{array}}{\Gamma \vdash fv : B[x \ / \ v]}\,\Pi\mathrm{e}$$

(c) Π-elimination rule

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\sum_{x:A} B(x)}\,\Sigma\mathrm{f}$$

(d) Σ-forming rule

$$\frac{\begin{array}{c} \Gamma \vdash a : A \\ \Gamma \vdash b : B[x \ / \ a] \end{array}}{\Gamma \vdash \langle a, b, \rangle : \sum_{x:A} B(x)}\,\Sigma\mathrm{i}$$

(e) Σ-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash v : \sum_{a:A} B(a) \\ \Gamma \vdash a : A, b : B[x \ / \ b] \vdash x : C \end{array}}{\Gamma \vdash \text{let } \langle a, b \rangle := v \text{ in } x : C}\,\Sigma\mathrm{e}$$

(f) Σ-elimination rule[4]

Figure 1: Inference rules for Π and Σ types

$$\Pi, \Sigma \underset{\text{Categorification}^5}{\overset{\text{propositional truncation}}{\rightleftharpoons}} \forall, \exists \tag{1}$$

2.1.1 Categorically

Dependent type theory is an extention on the simply typed lambda calculus. It is for locally cartesian closed categories, what STλC is for cartesian categories; the semantics of a dependent calculus will be in terms of some LCCC.

2.2 Polynomial functors

[2]TODO: Cite DM

[3]Some call this a 'dependent product', or 'dependent function', but this sometimes confuses people so I will simply call it either a Pi, or a forall.

[4]Technically this isn't the most general eliminator for Sigma's, as C can depend on the content of the type. For symetry and simplicity I omit this.

[5]Steve Awodey calls this 'categorification' [2], as it is only one direction of Curry-Howard. I know Equation 1 is just a more verbose way of writing 'Curry-Howard', but I find it useful to be verbose here.

A (multivariate) polynomial functor on set[6] is: a head-type, along with a (collection of) child family(ies) parameterised by the head. An object of a polynomial functor is a select head type, and the functions form the corresponding child(ren) parameterised by the head as seen in Equation 2[7]. Moraly polynomial functors can be thought of as types generic in some set of arguments, with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted.

$$(h : H) \times c_h \to \alpha \equiv \sum_{h:H} \alpha^{c_h} \tag{2}$$

Polynomial functors have all finite products and coproducts. Polynomial functors are also closed under (co)fixed points so I will write them using a notation a la inductives, I will use notation provided in [4]. For an explanation of the notation see Section 6.A.

2.2.1 Common pfunctors

2.2.1.1 Constant

```
inductive Const
    (T : Type) (a_1 ... a_n : Type) where
  | mk (t : T)
```

(a) Inductive Notation

$$(T : \text{Type}) \to \langle T, \lambda i.\lambda h.0 \rangle \tag{3}$$

(b) Polynomial

Figure 2: Constant pfunctor

2.2.1.2 Prj

2.2.1.3 Sum

```
inductive Sum (A B : Type) where
  | inl : A → Sum A B
  | inr : B → Sum A B
```

(a) Inductive Notation

$$\langle \mathbb{B}, \lambda\{0, \text{true } 0 \mapsto 1, \text{false } 1 \mapsto 1, _ \quad _ \mapsto 0\} \rangle \tag{4}$$

(b) Polynomial

Figure 3: Constant pfunctor

2.2.1.4 Option

2.2.1.5 Functions from fixed types

2.2.1.6 W

2.2.1.7 M

2.2.2 Lean formalization

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari

[6]For the readers familiar with category theory, simply think of categorical dependent products, and (co)limits when I use the set-theoretic language.

[7]More can be read on the ncatlab article [3], we restrict ourselves to Set, so I recommend skipping to that section.

voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

2.2.3 F-(co)algebras

2.3 Recursion and Inductives

We are all familiar with algebraic datatypes from OCaml. These are structures freely generated from a set of constructurs. The classic example is a list, given as two constructors, `cons : 'a → 'a list → 'a list` and `nil : 'a list`, and a way to consume lists `fold : 'a → ('b → 'a → 'a) → 'b list → 'a`. We write this as seen in Listing 1. This is what is known as an inductive datatype. Inductive datatypes are well-founded trees[8] any function that terminates, must by definition return at most a finite (but unbounded) amount of constructors. This means fold is guarangteed to terminate too. All (primitive) recursive functions can be compiled into a call to fold[9].

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b

type 'a list = Nil | Cons of 'a * 'a list
let nil = Nil
let cons h t = Cons(h,t)
let rec fold bh ih = function | Nil → bh | Cons(h, t) → ih h (fold bh ih t)
```

Listing 1: Definition of list in OCaml

2.3.1 Categorically

Given the family of endofunctors on **Set**. Consider the polynomial given by $L_A = 1 \oplus (\underline{A} \otimes \mathrm{id})$, or in its expanded form in Equation 5. Now recall the definition of the category of L_A-algebras [10]. The lists we are used to are exactly the initial object in this category. The initial map takes an algebra $f : L_A(B) \to B$ and yields a map $\mathrm{fold} : \mu L_A \to B$[11]. This is exactly what we want, as expanding the definition or $L_A(B)$ and distrubuting this over the arrows we get the signature above.

$$\begin{aligned} L_A(\quad X \quad) &= \quad 1 \quad \oplus \quad (A \quad \otimes \quad X) \\ L_A(\quad f \quad) &= (\iota_l \circ !) \oplus_m (\mathbb{1}_A \otimes_m f\ ) \end{aligned} \tag{5}$$

2.4 Corecursion and coinductives

Coinductives are the duals of inductives, they have an operation `unfold` which is the dual of the operation `fold`.

[8] OCaml is quite stupid and allows for non well-founded trees, an example is `let rec degen = Cons((), degen)`. This is because inductives and coinductives are 'coincident' in OCaml. We will come back to this in the next section.

[9] TODO: Cite McBride's work

[10] TODO: Cite algebra definition

[11] TODO: Cite Neel

2.4.1 Morally

Morally, if you have never seen `unfold`'s or coinductives, simply view it as if a fix F yields an X, then cofix F yields a lazy X[12]. We can think back to what we did in Foundations of Computer Science, when we made lazy lists we also had to change recursive occurances to be thunked, this thunking allows infinite structures to exist in finite memory.

2.4.2 In OCaml

```
type 'a colist = Nil | Cons of 'a * (() → 'a colist)
let nil = Nil
let cons h t = Cons(h,() → t)
let rec fold bh ih = function | Nil → bh | Cons(h, t) → ih h (fold bh ih t)
```

Listing 2: Definition of list in OCaml

2.4.3 Categorically

2.5 Tools used

2.6 Requirement Analysis

To expand on the success critera given in the proposal, I specified the project using MoSCoW. Here the success critera have become MUSTs, and improvements have become SHOULD. I have done this for the core and each of the extentions of the project.

2.6.1 State machine encoding (Core)

SM1 The SME MUST be implemented.

SM2 The SME MUST be constant time under destructuring.

SM3 The SME MUST be able to express the NT monad, Section 2.6.2.

SM4 The equivelence between SME and PA MUST be implemented.

SS1 The SME SHOULD have a coinduction principle.

SC1 The SME COULD have a productivity transform Section 2.6.5.

SC2 The SME COULD have an Interaction tree library Section 2.6.4.

SC3 The SME COULD have a Choice Tree library Section 2.6.6.

SW1 The SME WONT have a syntax macro a la [4].

2.6.2 Non-termination-monad (Core)

NM1

NS1

NC1

NW1

2.6.3 ABI Type (Extention)

2.6.4 Interaction Trees (Extention)

2.6.5 Futumorphic productivity (Extention)

2.6.6 Choice Trees (Extention)

[12] I. e. fix L_A gives lists, then cofix L_A gives lazy lists.

Chapter 3: Implementation

This chapter describes the implementation of the `Sme` constructs.

3.1 Transiteration

A family of functions, that keep solving problems throughout this dissertation are what I call transliterations. Given some parameter span X[13], either in some **Type**, or more commonly over some universe $\mathcal{U}$. We define a transliteration on X as a family of functions $t_{a,b} : Xa \to Xb$, such that they are closed under composition $t_{b,c} \circ t_{a,b} = t_{a,c}$[14], and the self-path is identificaion $t_{a,a} = \mathrm{id}$.

The trivial instantiation of a transliteration is a `cast`[15], here we pick X as equal types, obviously `cast aa = id` and `cast bc ∘ cast ab = cast (ab.trans bc)`. One could even say that a transliteration is a function that behaves like `cast`.

An example we will keep defining is universe transliteration, here we take $X = \mathrm{ULift}_{(-)}$, using this we define the transiteration as seen in Listing 3. This is the closest we get to having omega-types and cumilativity in lean; as long as the function is applied at a usage of ULift it all just works. We will use this function to define transliteration between universe-lifted polynomials, we will see more of this in Section 3.3.1.

```
universe u v w
variable {A : Type u} {x : ULift A} {a : A}

@[inline] def transliterate : ULift.{v} A → ULift.{w} A := ULift.up ∘ ULift.down

@[simp] theorem transliterate_idempotent
    : transliterate ∘ transliterate = transliterate (A := A) := rfl
@[simp] theorem transliterate_noop
    : transliterate x = x                                    := rfl
```

Listing 3: Transliteration between universes of types

Another major usage of a transiteration was used when defining the eliminator for `ABI`, I will discuss this more when we get to Section 3.6. This was where I first discovered transliteration, and made it possible to define a universe-polymorphic eliminator.

[13]This not necessarily a type, as lean does not have omega-types (Setω from Agda).
[14]This is very similar to saying it's an idempotent, but technically $t_{b,c}$ and $t_{a,b}$ arent the same function.
[15]Or if you read [1], this is called `transport`.

3.2 Stream implementation

As proving these equivleneces is extremely challenging I decidede I would start by implementing it for the special case of streams. Streams are the text-book coinductive datatype that most people know as mentioned in Section 2.3. Therefore I expected this to be an easy task to start with to test showing the equivilence.

First I set up a class of preobject:

3.3 Expanding the progressive approximation theory

During the pheasability assesment I noticed that, in the current formalised theory of polynomials, the statement wouldn't even type-check. This stemmed from a problem with the corecursive principle for the M type in the old implementation. corec : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{U}\} \to (g : \beta \to P(\alpha ::: \beta)) \to \beta \to M\alpha$[16] [5]. The problem here is that both α and β have to both reside in $\mathcal{U}$. Solving this is done through the next two sections.

3.3.1 Universe lifting of polynomial functors.

The main problem caused here comes from the fact that lean isnt cummulative. This means it is impossible to express a universe hetrogouns typevector. In other words $\alpha ::: \beta$ is only typable if $\alpha : \text{TypeVec}.\{\mathcal{U}\}n$ and $\beta : \text{Type } \mathcal{U}$. The natural way of solving this is using the supremum in universe levels you get from ULift : Type $\mathcal{U} \to$ Type $(\max \mathcal{U}\mathcal{V})$. This means we can have $\beta :$ Type $\mathcal{U}$ and $\alpha :$ Type $\mathcal{V}$, then ulift both of them to a common universe ULift $\alpha ::: $ ULift $\beta :$ TypeVec.$\{\max \mathcal{U}\mathcal{V}\}(n+1)$[17].

Noticable the next hurdle we encounter is that PFunctors are restricted to a universe level. Recall the definition from Section 2.2.2. Observe how for a MvPfunctor.$\{\mathcal{U}\}n$, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept ULift $\alpha ::: $ ULift β. If we do not add the ability to lift P, the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. Luckily lifting a PFunctor is relatively easy. We define it as ULift $P \triangleq \langle \text{ULift } P.1, \lambda x \mapsto \text{ULift } (P.2x) \rangle$. This works and now we can move on to our goal[18].

3.3.2 Generalizing the corecursor

Now with all the work in the previous section, by generalizing corec'[19], we can define corecU : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{V}\} \to (g : \beta \to \text{ULift } P(\text{ULift } \alpha ::: \text{ULift } \beta)) \to \beta \to M.\{\mathcal{U}\}\alpha$. Notably we are able to fit the object into $\mathcal{U}$ (as opposed to in the SME).

The expected diagram using corecU and dest commutes.

[16] https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFunctor/Multivariate/M.lean#L152-L154

[17] Note we overload ULift as a notation to refer to lifting TypeVecs as well

[18] TODO: Speak with JV / W to see if this might be done in the lit, Ex : Locally presentable and accessable categories Adameck roshiski

[19] Done in PR NUMBER

3.4 State machine encoding

Noting the definition of corecU, one might wonder if you could define M from first principles for this. The problem one encounters is one of universes. As seen in the definition above, if one were to define a type whos construcor is directly the corecU definiton, it would hold a β : Type $\mathcal{V}$. This then forces the object to reside in Type $\max \mathcal{U}(\mathcal{V}+1)$. This is a problem as one loses most closure results as you will be lifting more and more. The main beinifit from this is the performance aspect. Going from reading to a depth n, to a depth $n+1$ is not $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$. This will be seen in Section 4.1. We will henceforth refer to the datatype SME.PreM.

3.4.1 PreM

As we speak about in Section 2.2.3, the M Type is the terminal object in the category of coalgebras. We can see through reasoning (cumilatively) in this category that PreM is weakly terminal. Looking at this category we want to somehow force the incoming morphisms together. This corresponds exactly to quotienting, for this we will use Bisimilarity.

3.5 Interaction trees

Interaction trees (ITrees) are a coinductive datastructure detailed in [6].

3.6 The ABI Type

The problem the ABI type tries to tackle is one of abstracting the runtime datatype through functions. Given an isomorphism eq : $\alpha \cong \beta$ for some types α and β, my first try at solving this involved constructing an object ABI $\alpha\beta$, making the following commute:

$$
\begin{tikzcd}
 & \beta \arrow[dd, "\mathrm{mkB}"] \arrow[ddrr, "\mathbb{1}_\beta"] & & \\
\alpha \arrow[ur, equal, "\mathrm{eq}"] \arrow[r, "\mathrm{mkA}"] \arrow[ddr, "\mathbb{1}_\alpha"] & \mathrm{ABI}\ \alpha\beta \arrow[r, "\mathrm{destB}"] \arrow[dd, "\mathrm{destA}"] \arrow[dr, "\mathbb{1}_{\mathrm{ABI}\ \alpha\beta}"] & & \beta \arrow[dl, "\mathrm{mkB}"] \arrow[ddll, bend left, equal, "\mathrm{eq}"] \\
 & & \mathrm{ABI}\ \alpha\beta & \\
 & \alpha \arrow[ur, "\mathrm{mkA}"] & &
\end{tikzcd}
$$

Figure 4: Operations on ABI

Additionally I had an elimination principle satisfying the two equations below.

```
def elim : {motive : ABI _ _ eq → Type w}
        → (hLog : (z : A) → motive (mkA z))
        → (hCheap : (z : B) → motive (mkB z))
        → (eqA : ∀ z, hLog z ≍ hCheap (eq z))
        → (eqB : ∀ z, hCheap z ≍ hLog (eq.symm z))
        → (v : ABI _ _ eq) → motive v := _

theorem elimLog : {motive : carry → Type w}
        → {hLog : (z : A) → motive (mkA z)}
        → {hCheap : (z : B) → motive (mkB z)}
        → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
        → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
        → ∀ z, elim hLog hCheap eqA eqB (mkA z) = (hLog z) := _
theorem elimCheap : {motive : carry → Type w}
        → {hLog : (z : A) → motive (mkA z)}
        → {hCheap : (z : B) → motive (mkB z)}
        → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
        → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
        → ∀ z : B, elim hLog hCheap eqA eqB (mkB z) = (hCheap z) := _
```

Through quite a bit of work (which I call transliteration, as seen in the ABI file), You can free a universe level and open for a more general usage of the function.

3.6.1 Weak univalence

$$\begin{tikzcd}
\beta \arrow[rr, equal, "\mathrm{eq}"] \arrow[dr, "\mathrm{mkB}"'] & & \alpha \arrow[dl, "\mathrm{mkA}"] \\
& \mathrm{ABI}\ \alpha\beta &
\end{tikzcd}$$

Figure 5: Weak univalence up to shrink

3.6.2 Relation to Shrink and further universe transforming types

Chapter 4: Evaluation

4.1 Performance between SME and PA

After building the equivelence, and instantiating shrink, we now have the ability to compare the performance between 3 representations: The SME encoding (high performance, hpRuns), the PA encoding (slow, slRuns), and a hand made big implementation (bigRuns). We compare these 3 in Figure 6. In Figure 6 we see the 3 representations specalised to infinite streams, the code for which can be seen in Listing 4

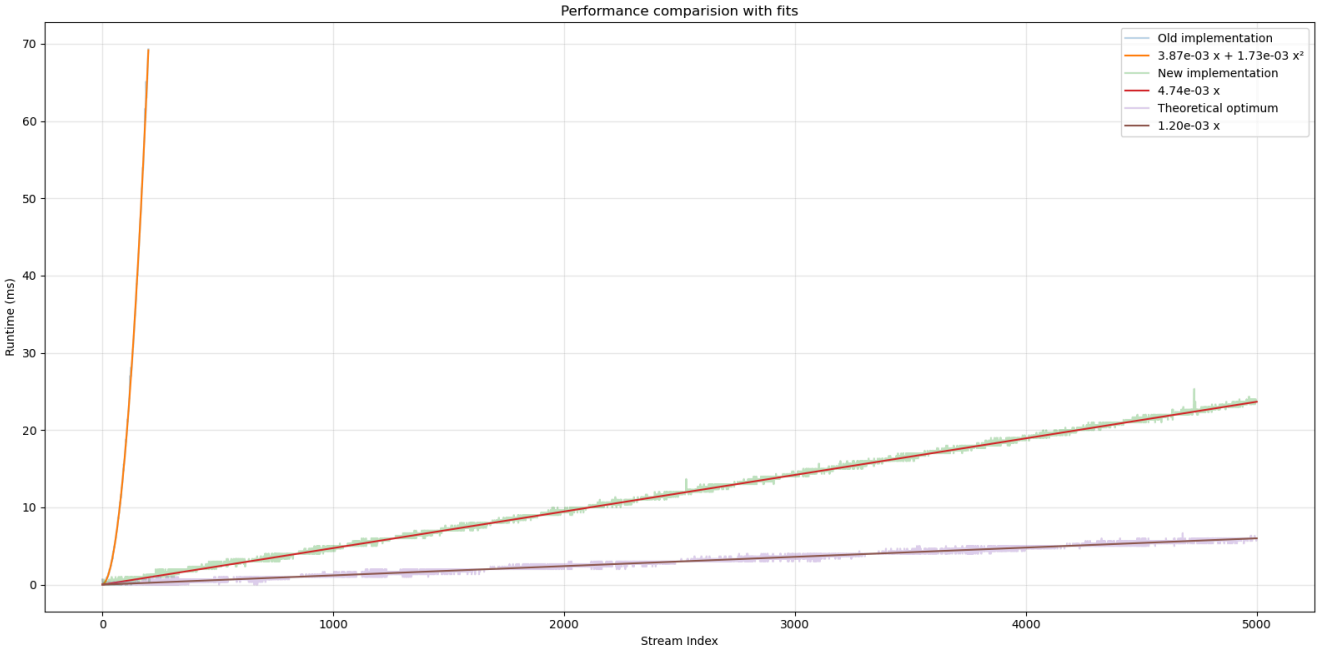


Figure 6: Performance of SME, PA, and Big representation

```
def QStream.Base : MvPFunctor 2 := ⟨
  Unit,
  fun _ _ ⇒ Unit
⟩

def QStreamSl α := M QStream.Base (fun _ ⇒ α)
def QStreamHp α := HpLuM QStream.Base (fun _ ⇒ α)

structure QStreamBig.{u} (α : Type _) where
  corec ::
    {t : Type u}
    (functor : t → Nat × t)
    (obj : t)

def numsSl : QStreamSl Nat :=
  .corec _ (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩)
  Nat.zero

def numsHp : QStreamHp Nat :=
  .corec' (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩)
  Nat.zero

def numsBig : QStreamBig Nat :=
  QStreamBig.corec (fun n ⇒ ⟨n, n + 1⟩) 0

def QStreamBig.getNth (x : QStreamBig Nat) : Nat → Nat
  | 0 ⇒ x.dest.fst
  | n+1 ⇒ x.dest.snd.getNth n

def QStreamSl.getNth (x : QStreamSl Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamSl.getNth (v .fz .unit) n

def QStreamHp.getNth (x : QStreamHp Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamHp.getNth (v .fz .unit) n
```

Chapter 5: Conclusions

Bibliography

[1] T. Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

[2] Steve Awodey, "Cambridge Lectures on Categorical Logic and Type Theory." https://awodey.github.io/cltt/, Mar. 2026.

[3] nLab authors, "polynomial functor." Jan. 2026.

[4] A. Keizer, "Implementing a definitional (co)datatype package in Lean 4, based on quotients of polynomial functors," Master's thesis, https://eprints.illc.uva.nl/id/eprint/2239/1/MoL-2023-03.text.pdf, 2023.

[5] The mathlib Community, "The Lean Mathematical Library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.

[6] L.-y. Xia *et al.*, "Interaction trees: representing recursive and impure programs in Coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.

6 Appendicies

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Quaerimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum idem fabellas Latinas ad verbum e Graecis expressas non inviti legant. Quis enim tam inimicus paene nomini Romano est, qui Ennii Medeam aut Antiopam Pacuvii spernat aut reiciat, quod se isdem Euripidis fabulis delectari dicat, Latinas litteras oderit? Synephebos ego, inquit, potius Caecilii aut Andriam Terentii quam utramque Menandri legam? A quibus tantum dissentio, ut, cum Sophocles vel optime scripserit Electram, tamen male conversam Atilii mihi legendam putem, de quo Lucilius: 'ferreum scriptorem', verum, opinor, scriptorem tamen, ut legendus sit. Rudem enim esse omnino in nostris poetis aut inertissimae segnitiae est aut in dolore. Omnis autem privatione doloris putat Epicurus terminari summam voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in voluptate aut a voluptate discedere.

Nam cum ignoratione rerum bonarum et malarum maxime hominum vita vexetur, ob eumque errorem et voluptatibus maximis saepe priventur et durissimis animi doloribus torqueantur, sapientia est adhibenda, quae et terroribus cupiditatibusque detractis et omnium falsarum opinionum temeritate derepta certissimam se nobis ducem praebeat ad voluptatem. Sapientia enim est una, quae maestitiam pellat ex animis, quae nos exhorrescere metu non sinat. Qua praeceptrice in tranquillitate vivi potest omnium cupiditatum ardore restincto. Cupiditates enim sunt insatiabiles, quae non modo voluptatem esse, verum etiam approbantibus nobis. Sic enim ab Epicuro reprehensa et correcta permulta. Nunc dicam de voluptate, nihil scilicet novi, ea tamen, quae te ipsum probaturum esse confidam. Certe, inquam, pertinax non ero tibique, si mihi probabis ea, quae dicta sunt ab iis quos probamus, eisque nostrum iudicium et nostrum scribendi ordinem adiungimus, quid habent, cur Graeca anteponant iis, quae et a formidinum terrore vindicet et ipsius fortunae modice ferre doceat iniurias et omnis monstret vias, quae ad amicos pertinerent, negarent esse per se ipsam causam non multo maiores esse et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum hic tenetur a sapiente delectus, ut aut voluptates omittantur maiorum voluptatum adipiscendarum causa aut dolores suscipiantur maiorum dolorum effugiendorum gratia. Sed de clarorum hominum factis illustribus et gloriosis satis hoc loco dictum sit. Erit enim iam de omnium virtutum cursu ad voluptatem proprius disserendi locus. Nunc autem explicabo, voluptas ipsa quae qualisque sit, ut tollatur error omnis imperitorum intellegaturque ea, quae voluptaria, delicata, mollis habeatur disciplina, quam gravis, quam continens, quam severa sit. Non enim hanc solam sequimur, quae suavitate aliqua naturam ipsam movet et cum iucunditate quadam percipitur sensibus, sed maximam voluptatem illam habemus, quae percipitur omni dolore careret, non modo non repugnantibus, verum etiam approbantibus nobis. Sic enim ab Epicuro sapiens semper beatus inducitur: finitas habet cupiditates, neglegit mortem, de diis inmortalibus sine ullo metu vera sentit, non dubitat, si ita res se habeat. Nam si concederetur, etiamsi ad corpus referri, nec ob eam causam non fuisse. – Torquem detraxit hosti. – Et quidem se texit, ne interiret. – At magnum periculum adiit. – In oculis quidem exercitus. – Quid ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo et gravissimas res consilio ipsius et ratione administrari neque maiorem voluptatem ex infinito tempore aetatis percipi posse, quam ex hoc facillime perspici potest: Constituamus aliquem magnis, multis, perpetuis fruentem et animo et attento intuemur, tum fit ut aegritudo sequatur, si illa mala sint, laetitia, si bona. O praeclaram beate vivendi et apertam et simplicem et directam viam! Cum enim certe nihil homini possit melius esse quam Graecam. Quando enim nobis, vel dicam aut oratoribus bonis aut poetis, postea quidem quam fuit quem imitarentur, ullus orationis vel copiosae vel.

6.A Graphical p-functors

An alternative notation for Equation 2 can be given as: for a multivariate polynomial functor $P\alpha_0...\alpha_i$, with head types as variants $h_0...h_j$, and children c_{i,h_j}, it would conventionally be written as Equation 6, I will display the hs and cs as in Figure 7. I write an object of this functor, picked at h_0 with functions f_i, as seen in Figure 8. The justification of this notation will be seen Section 2.2.2.

$$(j' : \text{Fin } j) \times \left(h : h_{j'}\right) \times (i' : \text{Fin } i) \to c_{i',h} \to \alpha_{i'} \tag{6}$$

$$\begin{array}{c|c|c|c}
\alpha_i & c_{i,v} & ... & c_{i,v} \\
\vdots & \vdots & \iddots & \vdots \\
\alpha_0 & c_{0,v} & ... & c_{0,v} \\
\hline
 & k_0\ (v : h_0) & ... & k_j\ \left(v : h_j\right)
\end{array}$$

Figure 7: The head and child components of $P\alpha_0...\alpha_i$

$$\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v}
\end{array}$$

$$\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots$$

$$k_0\ v$$

Figure 8: An object of $P\alpha_0...\alpha_i$

Mapping on MvPfunctors is done by composition of the arrows. This can be seen in Figure 9. By set forming a category, mapping a composition is the same as mapping each one by one.

$$\begin{array}{ccc}
\beta_i & \xleftarrow{g_i} & \alpha_i \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0
\end{array}
\qquad
\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v}
\end{array}
\qquad\qquad
\begin{array}{ccc}
\beta_i & \xleftarrow{g_i \circ f_i} & c_{i,v} \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0 \circ f_0} & c_{0,v}
\end{array}$$

$$\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots

6.A.A Common pfunctors

6.A.A.A Constant

A functor, fixed at a constant type T and a arity i, denoted $\text{const}_i T$, shown in Figure 10. It has one constructor $\text{mk}_\alpha : T \to \text{const}_i T \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{const}_i T \alpha_0 ... \alpha_i \to T$.

α_i	0
$\vdots$	$\vdots$
α_0	0
	$\text{mk}\ (v : T)$

Figure 10: Definition of $\text{const}_i T$

$$\begin{array}{ccc}
\beta_i \xleftarrow{g_i} \alpha_i & \alpha_i \xleftarrow{¡} 0 & \beta_i \xleftarrow{¡} 0 \\
\vdots \qquad \vdots & \vdots \qquad \vdots & \vdots \qquad \vdots \\
\beta_0 \xleftarrow{g_0} \alpha_0 & \alpha_0 \xleftarrow{¡} 0 & \beta_0 \xleftarrow{¡} 0 \\
\hline
\text{<\$>} & \text{mk } v & = \quad \text{mk } v \\
\hline
 & \text{mk}_\alpha v & \text{mk}_\beta v
\end{array}$$

Figure 11: By initial hom uniqueness,

the object is invariant under mapping.

6.A.A.B Prj

Given a typevector $\alpha_0 ... \alpha_i$, and a select index into the vector $(n : \text{Fin } i)$, the projection functor holds a value of type α_n. It is defined in Figure 12. It has one constructor $\text{mk}_\alpha : \alpha_n \to \text{prj}_n \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{prj}_n \alpha_0 ... \alpha_i \to \alpha_n$.

$$
\begin{array}{c:c:}
\alpha_i & 0 \\
\vdots & \vdots \\
\alpha_n & 1 \\
\vdots & \vdots \\
\alpha_0 & 0 \\
\cdots\cdots & \cdots\cdots \\
 & \mathrm{mk} \\
 &
\end{array}
$$

Figure 12: Definition of prj_n

$$
\begin{array}{ccccccc}
\beta_i & \xleftarrow{g_i} & \alpha_i & \alpha_i & \xleftarrow{¡} & 0 & \beta_i \xleftarrow{¡} 0 \\
\vdots & & \vdots & \vdots & & \vdots & \vdots \\
\beta_n & \xleftarrow{g_n} & \alpha_n & \alpha_n & \xleftarrow{\boldsymbol{K}v} & 1 & \beta_n \xleftarrow{\boldsymbol{K}(g_n v)} 1 \\
\vdots & & \vdots & \vdots & & \vdots & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0 & \alpha_0 & \xleftarrow{¡} & 0 & \beta_0 \xleftarrow{¡} 0 \\
\cdots & & & \cdots & & & \cdots \\
\text{<\$>} & & & \mathrm{mk} & & = & \mathrm{mk} \\
 & & & \cdots & & & \cdots \\
 & & & \mathrm{mk}_n v & & & \mathrm{mk}_n(g_n v)
\end{array}
$$

Figure 13: By initial hom uniqueness,

mapping only changes the nth argument.

6.A.A.C Sum

6.A.A.D Option

6.A.A.E Functions from fixed types

6.A.A.F W

6.A.A.G M

7 Proposal

See proposal overleaf

Efficient coinductives through state-machine corecursors

Supervisor: Alex Keizer

Marking supervisor: Jamie Vicary

Director of studies: Russell Moore

Word count: 993

1 Work to be undertaken

Previous work has shown how coinductive types can be encoded in Lean as quotients of polynomial functors (QPFs)[1]. This is done through progressive approximation (Section 4.1.2). For example accessing the n index of a stream takes $\mathcal{O}(n^2)$ time. This becomes an issue when using Lean as a general purpose programming language. An alternative approach is using a state-machine based encoding (Section 4.1.1) of cofixed points. With these we get nice computational behaviour of the compiled code. In Figure 1 we can see in blue the performance from the current implementation, versus in orange the performance of a prototype implementation the state-machine based version.

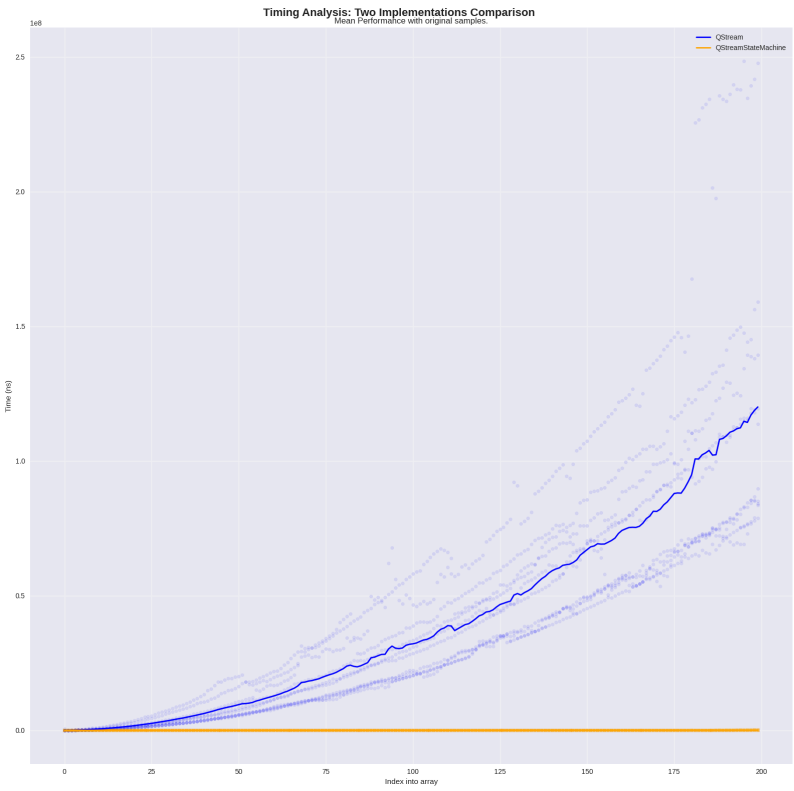


Figure 1: Graph plotting current performance v possible gains

2 Core project

This project will be implementing the state-machine encoding (Section 4.1.1) of coinductive types, and formalising the equivalence between these two embedding (Section 4.3). I will start with the special example of equivalence between `Streams` in the two representations, then building up to the general cofix structure.

After this I will be implementing a coinductive monad allowing us to encode non-termination as an effect using the state-machine representation. This will be an instance of the two representations.

3 Starting point

I have worked with QPFs on the meta-programming side for an internship between my Part Ia and Part Ib. During this I learnt of the basics of polynomial functors, and `TypeVec`s. This means I am aware of what the underlying structures are when it comes to the raw implementation. I have also done a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This lead to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge (#28112) an implementation of commutable Shrink to mathlib that later got reverted when it was found to be inconsistent.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I Include these for completeness and transparency.

4 Substance

Section 4.1.1, Section 4.3 and Section 4.4 are all discuss structures that will be implemented throughout the project. Section 4.1.2 and Section 4.2 both are implemented in Avigard et al [1].

4.1 The M type

The M type is the name given to the terminal coalgebra of polynomial functors; the possibly infinitely deep trees. This means the implementations must have both: A a corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructurer $\text{dest} : \text{M}F \to F(MF)$. We have two encodings of the M type.

4.1.1 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Then you quotient over bisimilarity, thereby only allowing direct usage of $\text{dest} : \text{Sme } F \to F(\text{Sme } F)$. The baseline implementation of this should be straight-forward.

4.1.2 Progressive approximation encoding

The other way is to generate them by progressive approximation where earlier trees must "agree" with the later ones. Agreement is given by them being the same up to the previous depth as seen in Figure 2.

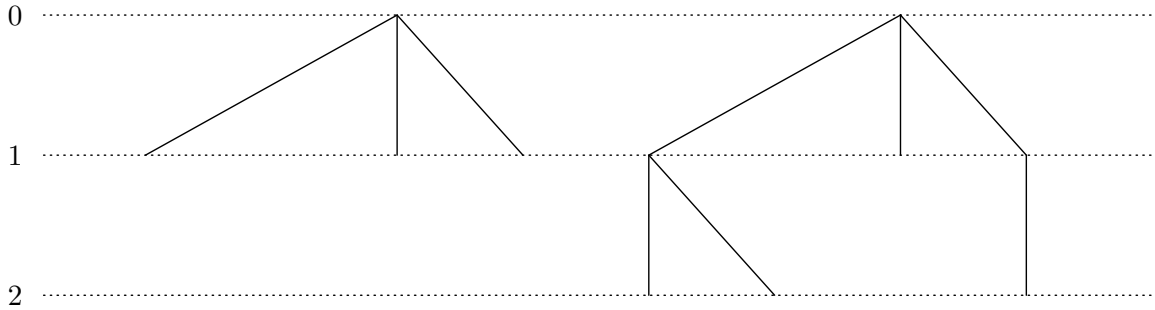


Figure 2: Agreement of two trees of height 1 and 2

4.2 The `Cofix` type

`Cofix` is the terminal coalgebra in QPFs, These are simply a quotient over M types lifting the quotient from the source QPF.

4.3 The equivalence

The equivalence between the current implementation of `Cofix` and M and the state-machine representation is the core of the project. The functions in both directions are given by $f : \mathrm{corec}_{\mathrm{Pae}}\mathrm{dest}_{\mathrm{Sme}}$ and $f^{-1} : \mathrm{corec}_{\mathrm{Sme}}\mathrm{dest}_{\mathrm{Pae}}$. The difficulty comes from proving $f \circ f^{-1} = \mathbb{1}$ and $f^{-1} \circ f = \mathbb{1}$. The proof of these equalities will be proven using bisimilarity, and other parts of the theory currently established for M types. Simpler versions of this equivalence also exist for the simpler representation.

4.4 The non-termination monad

The non-termination monad is a simple example of a coinductive. This will be useful for testing the state-machine representation's performance. The structure of the monad as a coinductive has two constructors as seen in this psudocode:

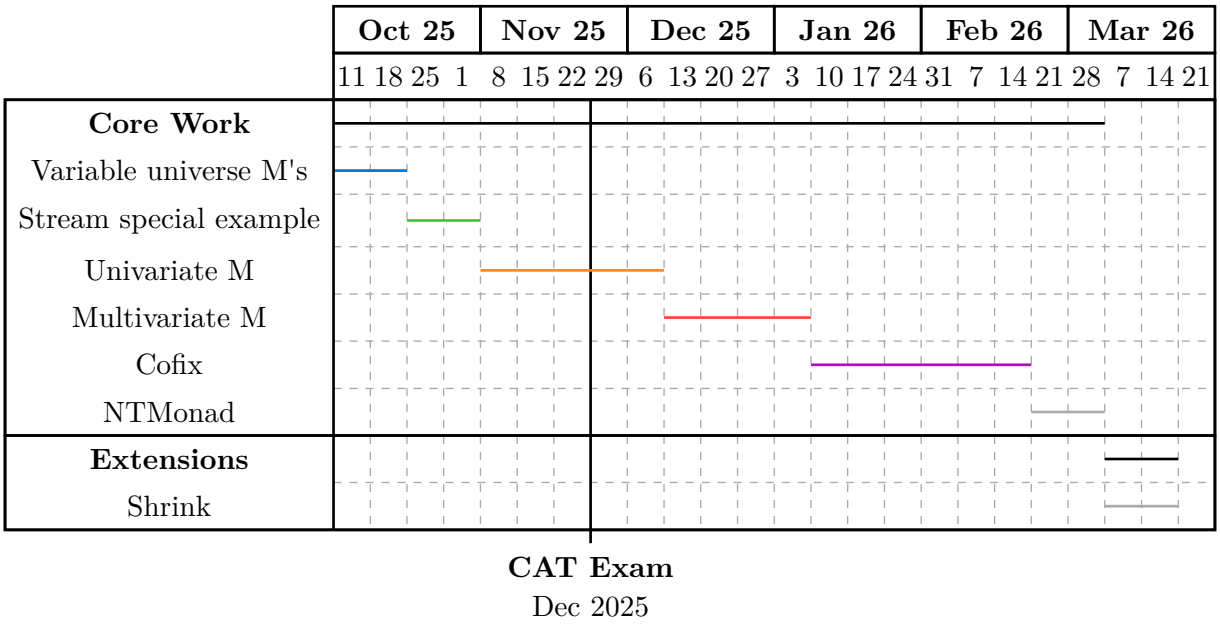
```
coinductive NTMonad (A : Type)
  | val : A -> NTMonad A
  | tau : NTMonad A -> NTMonad A
```

5 Evaluation

This projects success can be broken down into a few disjoint criteria.

1. If the equivalence (Section 4.3) has been proven,
2. how close to the theorised performance we get with the state-machine encoding,
3. and the ability to construct the example structures.

6 Core timeline



The plan for work would be divided into a few different stages.

6.1 Variable universe `Ms` (2025-10-11 2w 2025-10-24)

To begin with I have to make the corecursor universe polymorphic. As stated in Section 3 I have most of this code written. I will attempt to get this merged into mathlib, but if this review process takes too long I will be working on my own branch.

6.2 `Stream` special example (2025-10-25 2w 2025-11-7)

The first step would be implementing `Stream` under the two representations. Then I will familiarise myself with using bisimilarity to formalise the equivalence. This is a first step to just understand how these features work.

6.3 Univariate `M` (2025-11-8 2w 2025-11-21 + (CAT exam) + 2025-12-05 3w 2025-12-23)

Implementing the univariate `M` type is the natural next step from this. This will be the natural next step in difficulty. The work should lay the groundwork for the next proofs.

6.4 Multivariate `M` (2025-12-26 4w 2026-01-23)

The next step will be generalising the univariate implementation, the same structure of the proof should be the same for the multivariate case. The main difficulty here comes from working with `TypeVec`s. These are quite difficult to reason about.

6.5 `Cofix` (2026-01-24 6w 2026-03-06)

Finally, the `Cofix` type has to be implemented. This will be the most challenging part as I will have to work with quotients.

6.6 Implementing the NTMonad (2026-03-07 2w 2026-03-20)

This will be a demonstration of all of the previous work.

7 Extensions

7.1 Shrinking the representations (2026-03-21 2w 2026-04-04)

This involves finding a sound way allowing the current theory to use the efficient representation. The soundness of this will be difficult to reason about as it requires working with the Lean code generator. Therefore I leave this as an extension.

8 Resources

Access to the restricted side of the lab is needed for working with the greater team.

Bibliography

[1] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.